ServiX

Gestión de clientes, servicios y atención con IA

DOCUMENTACIÓN DE INTELIGENCIA ARTIFICIAL

Documentación del Asistente de IA de ServiX

Proveedor, arquitectura, flujo, contexto del sistema, parámetros, errores, límites y seguridad

Proyecto	ServiX — Gestión de clientes, servicios y atención con IA
Código del documento	DOC-08
Versión	1.0
Fecha	01/11/2026
Estándar de referencia	Documentación técnica del proyecto
Institución	Universidad de San Buenaventura
Asignatura	Optativa II
Docente	Ing. Jairo Armando Salcedo Aranda

Equipo de trabajo

Nicolas Quiroz Gomez	Product Owner
Juan Sebastian Garzon Suarez	Scrum Master
Juan Esteban Olaya Muñoz	Developer

Control del documento

Información general

Título	Documentación del Asistente de IA de ServiX
Código	DOC-08
Versión	1.0
Fecha de emisión	01/11/2026
Estado	Versión 1.0 emitida; aprobaciones pendientes
Proyecto	ServiX
Autor	Equipo ServiX
Clasificación	Uso académico; sin información confidencial

Historial de revisiones

Versión	Fecha	Descripción del cambio	Autor
0.9	01/11/2026	Borrador inicial en formato plano (archivado como respaldo v1).	Equipo ServiX
1.0	01/11/2026	Rediseño completo: portada, control de versiones, numeración, figuras y tablas rotuladas, mayor profundidad.	Equipo ServiX

Aprobaciones

Rol	Nombre	Fecha	Firma / estado
Elaboró	Juan Esteban Olaya Muñoz (Developer)	Pendiente	Pendiente
Revisó	Juan Sebastian Garzon Suarez (Scrum Master)	Pendiente	Pendiente
Aprobó	Nicolas Quiroz Gomez (Product Owner)	Pendiente	Pendiente

Las aprobaciones se encuentran **pendientes**: no se registran firmas ni fechas de aprobación hasta que los responsables las emitan.

Tabla de contenido

1 Introducción **6**
2 Arquitectura **7**
3 Flujo de una consulta **9**
3.1 Clasificación de intención 9
3.2 Prioridad del escalamiento 10
4 Contexto del sistema (prompt) **11**
5 Parámetros de configuración **12**
6 Manejo de errores **13**
7 Límites, costos y seguridad **14**
7.1 Límites 14
7.2 Seguridad y privacidad 14
8 Qué falta verificar **15**

Lista de figuras

Figura 1. Secuencia del chat con IA y del escalamiento 7

Lista de tablas

Tabla 1. Resumen del asistente 6
Tabla 2. Componentes del asistente 8
Tabla 3. Intenciones y palabras clave (resumen) 9
Tabla 4. Reglas de prioridad 10
Tabla 5. Bloques del contexto del sistema 11
Tabla 6. Parámetros del modelo 12
Tabla 7. Errores y comportamiento 13
Tabla 8. Medidas de seguridad del asistente 14
Tabla 9. Verificación pendiente de la IA 15

1 Introducción

ServiX incluye un asistente conversacional ("Agente Virtual Inteligente de ServiX") accesible desde la pestaña **Chat IA** y desde el Portal del cliente. Responde preguntas sobre los servicios del usuario, orienta al administrador y, cuando el mensaje lo exige, escala el caso a una persona creando una solicitud de atención.

> **Advertencia. Estado de verificación**
> El flujo de IA está implementado y compilado, pero **la comunicación real con Gemini no se ha verificado** en dispositivo (casos CP-038, CP-039 y CP-048, no ejecutados). Este documento describe el comportamiento según el código.

Tabla 1. Resumen del asistente

Aspecto	Detalle
Proveedor	Firebase AI Logic con Gemini Developer API.
Modelo	`gemini-2.5-flash`.
Temperatura	0,7.
Máximo de tokens de salida	800.
Historial	Últimos 6 mensajes.
Credenciales	En el proyecto Firebase; no hay clave en el APK ni en el repositorio.
Clasificación de intención	Por palabras clave, antes de consultar al modelo.
Escalamiento	Solo para usuarios con rol cliente.
Interfaz del dominio	`AiChatRepository.getAiResponse(userMessage, history, systemContext): Result<String>`.

2 Arquitectura

La integración respeta las capas de la aplicación: la pantalla `ChatScreen` invoca a `MainViewModel`, que llama a `SendAiChatMessageUseCase` (dominio). Este depende de la interfaz `AiChatRepository`, implementada por `GeminiAiChatRepository` (datos). `AppContainer` crea el repositorio de forma perezosa.

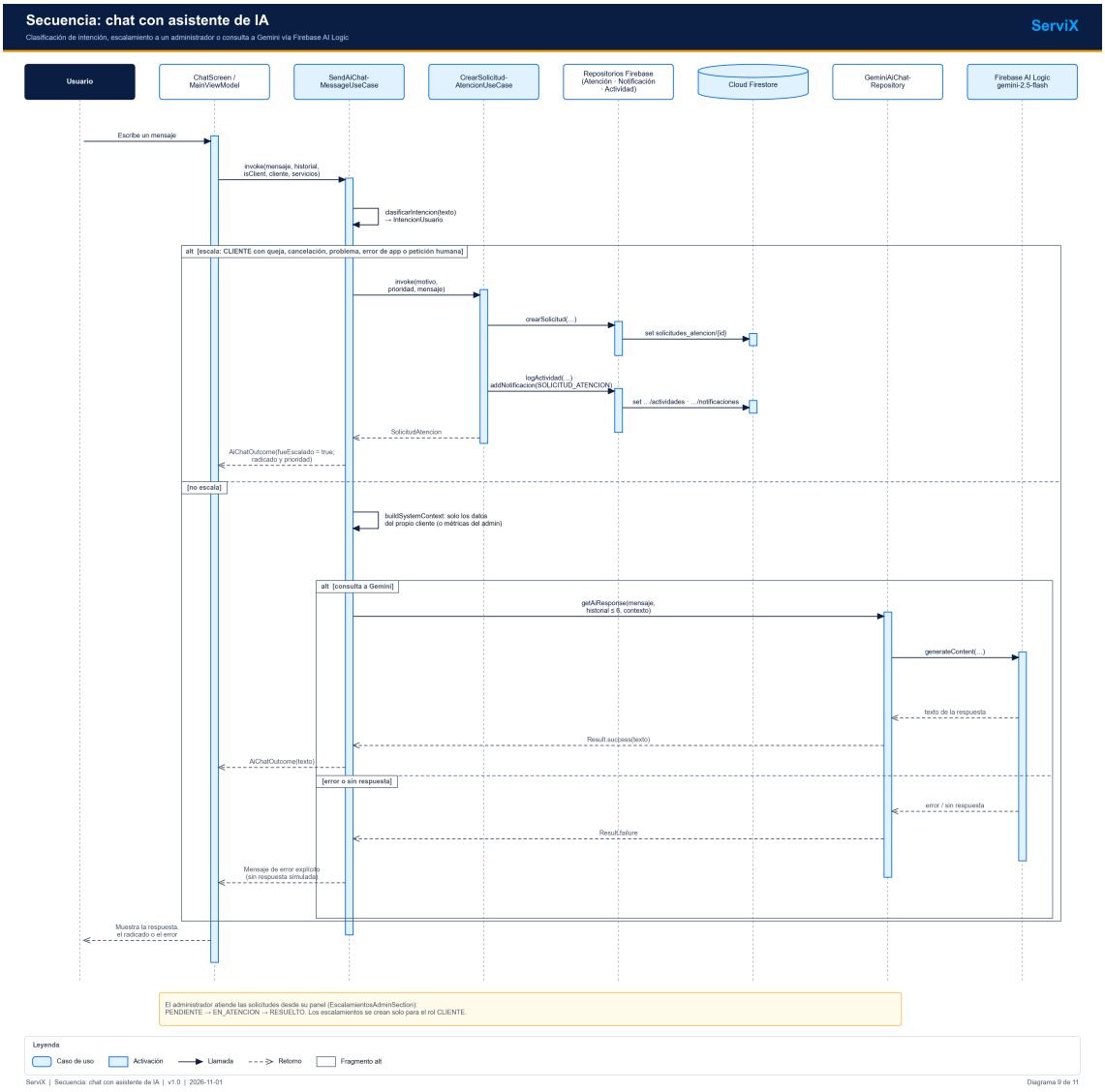


Figura 1. Secuencia del chat con IA y del escalamiento

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

Tabla 2. Componentes del asistente

Componente	Capa	Responsabilidad
`ChatScreen`	Presentación	Muestra los mensajes, el indicador de carga y el estado del chat.
`MainViewModel.sendChatMessage`	Presentación	Añade el mensaje, bloquea el reenvío mientras carga y publica la respuesta.
`SendAiChatMessageUseCase`	Dominio	Clasifica, decide el escalamiento y construye el contexto.
`CrearSolicitudAtencionUseCase`	Dominio	Crea la solicitud, registra la actividad y genera la notificación.
`AiChatRepository`	Dominio	Contrato del proveedor de IA.
`GeminiAiChatRepository`	Datos	Llamada al modelo mediante Firebase AI Logic.

3 Flujo de una consulta

1. El usuario escribe un mensaje; el ViewModel lo añade a la conversación y activa el indicador de carga.
2. El caso de uso recorta el texto y clasifica la intención por palabras clave.
3. Si el usuario es cliente y la intención es de escalamiento (solicitud de persona, queja, cancelación, problema o error de la aplicación), crea una solicitud de atención y responde con un mensaje que incluye prioridad, motivo y radicado; **no consulta al modelo**.
4. En caso contrario construye el contexto del sistema (rol, datos del usuario, servicios propios o métricas del negocio).
5. `GeminiAiChatRepository` crea el modelo con la configuración y la instrucción de sistema, inicia el chat con los últimos 6 mensajes (descartando los iniciales que no sean del usuario) y envía el mensaje.
6. Si el modelo devuelve texto, se muestra. Si falla o responde vacío, el caso de uso muestra un mensaje de inconveniente temporal que sugiere reintentar o solicitar un asesor.

3.1 Clasificación de intención

Tabla 3. Intenciones y palabras clave (resumen)

Intención	Ejemplos de palabras clave	¿Escala?
Solicitud humana	hablar con una persona, asesor, humano, administrador, vendedor, operador	Sí
Queja	queja, reclamo, inconforme, mal servicio, pésimo, denuncia, demanda	Sí
Cancelación	cancelar, anular, dar de baja, reembolso, devolución	Sí
Problema	problema, falla, daño, retrasado, tarde, no han llegado, no vinieron, cobro incorrecto	Sí
Error de la aplicación	error en la app, no carga, pantalla negra, se cierra	Sí
Consulta de estado	estado, cómo va mi, en qué estado, cuándo terminan	No
Consulta de servicio	mi servicio, mis servicios, mi orden, solicitud	No
Nueva solicitud	nuevo servicio, solicitar servicio, agendar, cotizar, cuánto cuesta	No
Consulta general	horario, contacto, garantía, teléfono, dirección	No
Otro	Cualquier otro texto	No (salvo frases como "hablar con alguien" o "inconformidad")

El orden de evaluación importa: la solicitud humana se evalúa primero y la queja antes que la cancelación. La clasificación es textual y no distingue negaciones ni sinónimos no listados; esto se registra como limitación.

3.2 Prioridad del escalamiento

Tabla 4. Reglas de prioridad

Condición	Prioridad
El texto contiene urgente, inmediatamente, grave, emergencia, falla crítica o cobro indebido	Urgente
La intención es queja, cancelación o problema, o el texto contiene retrasado o inconforme	Importante
Cualquier otro caso de escalamiento	Normal

4 Contexto del sistema (prompt)

El contexto que recibe el modelo como instrucción de sistema se compone de bloques definidos en `buildSystemContext`. No contiene credenciales. Su estructura es:

Tabla 5. Bloques del contexto del sistema

Bloque	Contenido
Identidad	Se presenta como el Agente Virtual Inteligente de ServiX, un servicio del sistema, no un usuario.
Tono	Responder siempre en español, de manera profesional, empática, cortés y concisa.
Regla estricta	No inventar precios, datos de clientes ni estados de servicios; admitir con transparencia lo que no se conoce.
Rol cliente	Nombre y correo del cliente; instrucción de privacidad absoluta (no mencionar datos de otros clientes ni del negocio); lista de **sus** servicios (nombre, estado, prioridad, categoría); explicación de los estados; datos necesarios para agendar un servicio.
Rol administrador	Nombre y empresa del perfil; métricas del sistema (clientes totales, servicios totales, pendientes, en proceso y completados); orientación sobre gestión de órdenes y atención.

Los servicios del cliente se filtran en el dispositivo: se incluyen únicamente los que coinciden con su identificador o con su correo. Esto limita la información enviada al proveedor, pero implica que los datos del usuario salen del dispositivo hacia el servicio de IA.

5 Parámetros de configuración

Tabla 6. Parámetros del modelo

Parámetro	Valor	Ubicación en el código
Backend	`GenerativeBackend.googleAI()`	`GeminiAiChatRepository`
`modelName`	`gemini-2.5-flash`	Constante `MODEL_NAME`
`temperature`	0,7	`generationConfig`
`maxOutputTokens`	800	`generationConfig`
Historial (`MAX_HISTORY`)	6 mensajes	Constante en el repositorio
`systemInstruction`	Texto del contexto	Argumento `systemContext`

Una temperatura de 0,7 favorece respuestas naturales sin ser excesivamente aleatorias; el límite de 800 tokens controla la longitud y el costo; el historial de 6 mensajes acota el contexto enviado en cada llamada.

6 Manejo de errores

Tabla 7. Errores y comportamiento

Situación	Comportamiento	Observación
Excepción del SDK o falta de red	Se registra y se devuelve `Result.failure`.	El usuario ve el mensaje de inconveniente temporal.
Respuesta vacía del modelo	Se convierte en fallo ("Respuesta vacía del modelo").	Igual que el caso anterior.
Mensaje en vacío	`MainViewModel` lo ignora.	No se envía nada.
Mensaje enviado durante una espera	Se ignora mientras `isChatLoading` es verdadero.	Evita llamadas duplicadas.
Caso de uso no disponible	El ViewModel muestra un mensaje de inconveniente.	Solo si no se inyectó el caso de uso.

Nota. Sin respuestas simuladas
La versión original devolvía respuestas predefinidas como si fueran de la IA cuando la API fallaba (hallazgo H-03). Ya no ocurre: el error se propaga. El texto de error mostrado al usuario es genérico y está redactado como un mensaje del asistente, por lo que no distingue entre fallo de red, de cuota o de configuración.

7 Límites, costos y seguridad

7.1 Límites

- Cuota y disponibilidad de Gemini Developer API sujetas al plan del proyecto Firebase.
- Respuestas de hasta 800 tokens y contexto de 6 mensajes; las conversaciones largas pierden contexto antiguo.
- El historial del chat vive en memoria del ViewModel: no se persiste y se pierde al cerrar la aplicación.
- La clasificación por palabras clave puede producir falsos positivos y negativos.
- El modelo puede equivocarse; la regla de no inventar datos reduce, pero no elimina, el riesgo.

7.2 Seguridad y privacidad

Tabla 8. Medidas de seguridad del asistente

Medida	Estado
Sin clave de IA en el APK ni en el repositorio.	Implementada.
Filtrado de los servicios del cliente antes de construir el contexto.	Implementada.
Instrucción de privacidad absoluta en el contexto del cliente.	Implementada (depende del modelo).
Límite de longitud y de historial.	Implementada.
App Check para limitar el uso de la IA a la aplicación legítima.	**Pendiente** (dependencia añadida, sin proveedor).
Cuotas y alertas de costo en la consola de Firebase.	**Pendiente** de configurar.
Aviso al usuario sobre el envío de sus datos a un servicio de IA.	No implementado.

8 Qué falta verificar

Tabla 9. Verificación pendiente de la IA

Verificación	Caso	Estado
Respuesta de `gemini-2.5-flash` a una consulta general con AI Logic activo	CP-038	No ejecutado
Escalamiento por queja que crea la solicitud y notifica	CP-039	No ejecutado
Mensaje de error sin respuesta simulada al fallar la red	CP-048	No ejecutado
Prueba automática del caso de uso con un repositorio de IA simulado	(sin caso)	No existe
Comunicación real con Gemini desde un dispositivo	—	No ejecutado

Recomendación: añadir pruebas unitarias de `SendAiChatMessageUseCase` con un repositorio de IA simulado (clasificación, prioridad, escalamiento y construcción del contexto) para cubrir la lógica sin depender de la red.